

Secure and Extensible Remote System Administration via Large Language Model Agents using the Model Context Protocol

Arya Jalindar Kadam

Department of Information Technology
BRAC's Vishwakarma Institute of Information Technology
Pune, India
aryakadam348@gmail.com

Prasad Rajaram Kute

Department of Information Technology
BRAC's Vishwakarma Institute of Information Technology
Pune, India
prasadmute0708@gmail.com

Hanmant Magar

Department of Information Technology
BRAC's Vishwakarma Institute of Information Technology
Pune, India
hanmant.magar@viit.ac.in

Tejaswini Jaywant Durge

Department of Information Technology
BRAC's Vishwakarma Institute of Information Technology
Pune, India
tejaswinidurge41@gmail.com

Soham Atul Deshmukh

Department of Information Technology
BRAC's Vishwakarma Institute of Information Technology
Pune, India
sohamatuldeshmukh2003@gmail.com

Abstract—The developed system allows remote management of computers using simple text messages sent through Telegram. At its core, a large language model (LLM) agent interprets these plain-language commands. An LLM agent sits behind the chat, figures out what the user wants, and executes it on the host through a local FastMCP server. The Model Context Protocol (MCP) ties these pieces together that gives a structured, schema-direct linkage of these elements by doing away with the need for numerous custom integrations. Knowing that it is dangerous to entrust an AI to do something, shell commands, the solution that is implemented is composed of a robust three-tier safety system. To begin with, a per-user whitelist of permissions, protects the issuing of commands by authorized people. Second, a blacklist is a regular expression based blacklist that proactively identifies and prevents possible harmful commands, like ‘rm -rf’, before they reach the operating system. Finally, a pattern filter scrutinizes the LLM-generated Python code and prevents the inclusion of dangerous imports.

The proposed solution agent was constructed using LangGraph and has been successfully tested with more than ten different models from major providers like Google, OpenAI, and Anthropic. Notably, switching between these AI models did not require modifications to the system’s toolset. In performance evaluations, the system achieved a 92 % success rate in 50 administrative tasks, effectively neutralized all 30 simulated adversarial attacks and delivered responses in an average of less than four seconds. This system has been in continuous operation since early 2026.

Index Terms—LLM, MCP, Remote Administration, Sandboxing, FastMCP, Telegram Bot, LangGraph, Multi-Model, ReAct Agent, RCE Mitigation.

I. INTRODUCTION

Both methods work,. They require the user to know exactly what command to type or what buttons to click. A junior admin who asks ”show me which process is using all the RAM” won’t get an answer from a bash prompt. This gap can be addressed with Large Language Models or LLMs. They convert requests to specific shell commands interpret the output and make decisions on what to do next [1], [2].

It is not getting LLMs to reason: it is to get it set up to operate safely. Every tool, like a file manager or package installer needs code to work with LLMs. The way each new LLM model works presents a big integration problem [3], [4]. There is a concern of safety. LLMs are able to invent things. Bad input can mislead them. On one bad reply a disk can be destroyed by an LLM [5], [6].

The two problems are solved by proposed system. We used Anthropic’s Model Context Protocol [7]. It is a standard which only allows each tool to be used with a single adapter and any standard model can be used. The user interface is a Telegram bot [8]. We decided on Telegram since it already supports logins and notifications, on all platforms. We added three security measures between the output of the LLMs and the kernel of the computers: user permissions, a list of banned commands and a filter that verifies the commands.

Part II deals with work. Part III-V discusses our objectives, the mechanism of our system and the way we did it. Section VI presents the findings of 50 tasks and 30 tests that are aimed

at deceiving the system. In Section VII and VIII, there are discussions as to what worked not and what we intend to fix.

II. LITERATURE REVIEW

The design was influenced by three bodies of previous literature: the way in which LLM agents perform system tasks, the way in which MCP is trying to standardise tool access, and the way in which sandboxes prevent the generated code to create damage.

A. LLM Agents for System Tasks

The latest survey lists than a hundred papers on LLM-based agents [1]. Most of these papers are based on some variation of the loop that Yao et al. Talked about which is called ReAct. This loop is simple: generate a plan execute a step look at what happened and then adjust [2]. Something that really struck us was the experiment that Cao et al. Did. They told GPT-4 to do 150 Linux administrator jobs like cleaning up disks setting up firewall rules and doing package upgrades. The model did the rest on its own [9].

Other people have used the idea to control engineering [10], [11] and multi-model pipelines. For example one LLM can send work to specialist models [12], [13]. This shows that LLM-based agents can reason. What we needed to make LLM-based agents work in life was a safe way to connect them to a live operating system, like Linux. We needed this to make LLM-based agents go live.

B. The Model Context Protocol

Prior to MCP, to bind a tool to an LLM, one needed to write glue code to the particular model calling convention. Models trained by Schick et al. to invoke APIs were trained on a small number of demonstrations to guide the process of API selection [3]; Patil et al. fine-tuned a single model to select APIs in particular to the task of API selection [4]. Both enhanced accuracy, yet all the new tools still required a new adapter. MCP [7] addresses the issue on the wire level, taking the client,server split of IDEs in the Language Server Protocol [14]. A tool publishes a name, a short description, and a JSON Schema describing its arguments; the LLM fills in the schema, and the MCP server runs the call over JSON-RPC 2.0 [15] (stdio for local use, HTTP+SSE for remote). Practically this implied that when we introduced a sixth tool in the middle of development, we wrote a single Python class and then restarted—there was no longer any change.

C. Sandboxing AI-Generated Code

An LLM with the ability to call the function of executing code, has arbitrary code execution - whether it wants it or not. It uses prompt-injection attacks: Liu et al. demonstrated that the apps LLM would run attacker payloads by masking instructions in user input and got them to do so successfully, and Greshake et al. demonstrated the same indirectly through documents retrieved by them, both by doing it successfully. Testing ChatGPT plugins, published by Iqbal et al., questioned more than forty-five percent of them, enabling them to send

and obtain data without a strong data-handling and permission checkup [16]. OWASP has now placed prompt injection as the highest risk of LLM applications [6].

The extent to which isolation is sufficient is based on the threat model. Docker containers are inexpensive and use the same host kernel [17]. At almost native speed, WebAssembly provides memory safety [18], [19]. Firecracker micro-VMs can boot to a minimal kernel in less than 125 ms per workload (as of 2020) [20] at the cost of operational load. We decided on the least heavy one, which is run on the host, but filter all shell commands by a regex blacklist and all Python snippets by pattern-based imports. Section V is the answer to the question why, and where we discovered it breaking.

III. OBJECTIVE

The primary objectives of this study are as follows:

- 1) **Unified protocol design:** To establish a common protocol that consolidates diverse functionalities such as shell operations, file management, Python execution, and system monitoring within the framework of the Model Context Protocol (MCP) [7]. The architecture is also made to be independent of the large language model (LLM) it uses, such that changing the model does not necessitate changes to the implementations of the tools. Several models with various providers were used to assess this property.
- 2) **Reduced system requirements:** To enable user interaction solely through Telegram, thereby avoiding the need for additional configurations such as virtual private networks (VPNs), secure shell (SSH) credentials, or dedicated software installations.
- 3) **Secure and fail-safe execution:** To incorporate automated safeguards that ensure reliable system operation without continuous human oversight. This is achieved through layered security mechanisms, including user access control via whitelisting, command validation using regular expression filtering, and pattern-based screening of Python code.
- 4) **Modular extensibility:** To design the system in a modular manner that simplifies the integration of new functionalities, such as database connectors or GitHub adapters, requiring minimal development effort—typically limited to implementing a single Python class followed by a system restart.

IV. ARCHITECTURE

The suggested system consists of three main elements, as it is depicted in Fig. 2. The startup of these components and their interaction with each other is handled by a central controller class, which is called RemoteAgentApp.

A. Telegram Bridge

The Telegram bridge, implemented in `telegram_bridge.py` (approximately 800 lines of code) is our primary entry point for all incoming user messages. It cross-references the Telegram user ID of the

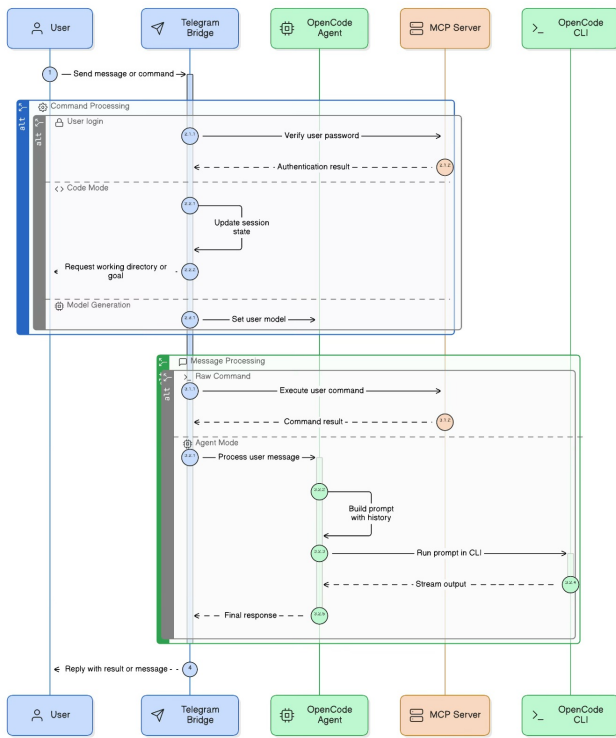


Fig. 1. Architecture Flowchart: a Telegram message enters the Bridge, is routed to the LLM Agent, which issues JSON-RPC tool calls to the MCP Server. Responses travel the same path in reverse.

sender with a YAML whitelist before determining the destination: explicit commands such as `/shell/` are sent directly to the MCP server; everything else is sent to the LLM agent.

The bridge eventually expanded to perform routing as well. Through an inline keyboard (`/models`), users can change LLM models in the middle of a conversation. Additionally, a dedicated Code Mode facilitates extended AI-assisted programming interactions (see Section V-C). Destructive tasks such as the deletion of files need a confirmation tap which expires after 60 seconds. Users that do not have a whitelisted ID can still use a password to log in, we added this feature after realising that we were continually locking ourselves out when logging in on new devices.

Furthermore, to improve accessibility, an alternative authentication mechanism has been incorporated. Users whose Telegram IDs are not present in the whitelist can authenticate using a password-based login to prevent unintended access issues during multi-device usage and testing scenarios.

B. LLM Agent

Our initial outlay was to an external CLI tool to do the LLM reasoning. This was until we had to support per-user context and model switching, when we removed it and substituted it with LangGraph [21], a graph-based agent framework that runs on LangChain [22]. The core is a `create_react_agent` graph: the model examines the

conversation, selects a tool, reads the output and repeats until it has an answer [2]. We wrapped each of our five tools in a LangChain `StructuredTool` with a Pydantic schema so the LLM gets typed arguments instead of raw strings, this in itself removed a category of parsing errors that we were experiencing.

Creation of the models passes via a factory in `providers.py`. You feed it a string such as `google/gemini-2.5-flash` and it returns a ready-to-use instance of LLM. Right now application supports six Gemini models (Flash is the default), four OpenAI models and Anthropic Claude. Switching occurs at runtime via Telegram with `/models`; the tools are unaware of or concerned with what model is invoking them.

Each user gets a `UserContext` which is basically a sliding window of the last 30 messages. Trimming it re-grounds us to the initial `HumanMessage`; otherwise Gemini complains of orphaned tool calls. Each agent invocation is enclosed in `asyncio.wait_for` (120 seconds by default, 600 in Code Mode) to ensure that a stuck model does not prevent the bot.

C. MCP Server

The FastMCP is based on the server (`mcp_server.py`), which is used to host all the tools. Three conditions must be met before a call is executed, namely: the permission tier of the caller must include the requested tool, the command must pass the regex blacklist, and (if it is Python code) the import filter must approve the request. In case of failure of any of the checks, the call is terminated and the admin receives a Telegram message. We also hard-block access to the `.env` at three different locations (path validation, shell filter, Python filter) following an initial scare where the LLM was attempting to `cat` our API keys to “check the configuration.”

D. Use-case

Fig. ???. Based on their roles and permissions, this use case diagram shows how different types of users use a system. The process starts with different types of users, such as unauthorized users, read-only users, standard users, and admin users. The authentication step is the first thing that all users must do. This is where the system checks their identity, like by using a Telegram ID. The system immediately rejects the connection if the user is not authorized, which keeps things safe.

Users can do things based on their role once they have been verified. Users who can only read can only do things that don’t change the system, like reading server logs or querying system telemetry. Standard users can do things like run safe shell commands or sandboxed Python scripts, which are normal tasks. These operations are watched so that any harmful or destructive commands can be found and stopped before they are run. Admin users have the most access, which lets them run harmful commands and change user permissions.

All of these operations eventually interact with the host system, where things like input/output operations and sub-

process execution happen. The diagram shows a clear role-based access control system that keeps sensitive operations to authorized users only, which keeps the system safe and sound.

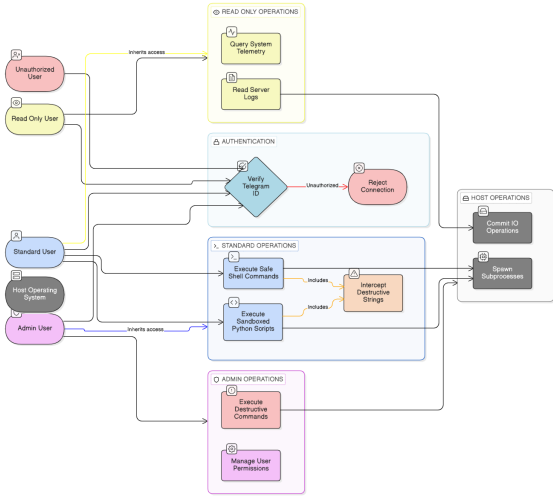


Fig. 2. Use-case Diagram

V. METHODOLOGY

A. Tool Suite

We shipped what is listed in Table I. All tools are Python classes, named, with a JSON Schema of required inputs, and a required permission level. The schema is simply seen by the LLM - it is not aware of the implementation.

TABLE I
FASTMCP TOOL SUITE

Tool	Capability	Min. Tier
ShellTool	Run OS commands, capture stdout/stderr	user
FilesTool	List, read, write, delete files	user
PythonExecTool	Execute Python snippets on host	user
AppTool	Launch or kill desktop applications	user
SystemTool	CPU, RAM, disk, process list	readonly

B. Data Flow

Fig. 3 shows a safe system that uses an AI model to process a user's request and carry it out safely. The end user sends an input first. This input goes to the Identity and Authentication stage (P1), where the system uses a configuration file to check the user's identity and permissions. Once the user is verified, the request goes to Semantic Parsing (P2), where the system figures out what the user wants and turns it into a structured format using stored context or memory. The LLM provider then gets this structured request and makes instructions or code to carry it out.

Before execution, the Sandbox Interceptor (P3) checks and cleans up the code to make sure it doesn't do anything harmful or unauthorized. If the code is not safe, a security exception is raised and sent right away to be handled. If it is safe, it goes to the FastMCP Execution stage (P4), where the code runs on the

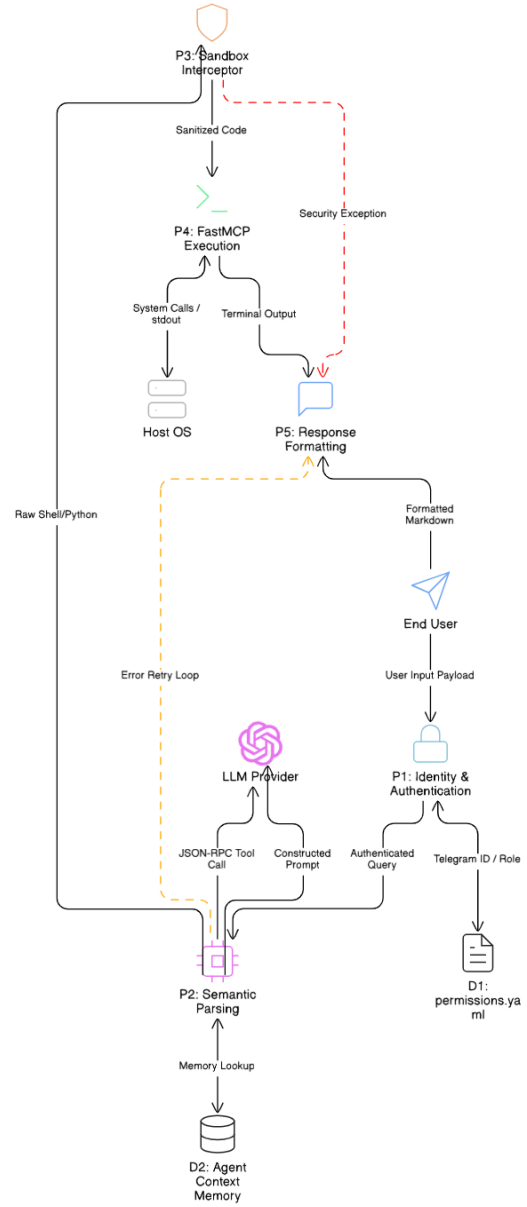


Fig. 3. Data flow

host system and gives output. The Response Formatting stage (P5) then gets this output and changes it into a format that is easy for the user to read. The user then gets the formatted result back. The system also has an error retry loop, which means that if something goes wrong during processing, it can try again with the AI model to get a better result.

C. Code Mode

Single-turn administration commands are suitable in the case of "checking disk space", but not in the case of refactor the logging module. We added there Code Mode. The user writes the command: `/code` and chooses a working directory, describes the purpose and the bridge switches to: STATE

CODING. The agent in this state receives a different system prompt - it is forced to follow an Explore → Plan → Execute → Verify sequence as opposed to attempting to solve everything on a single attempt. We increased the iteration limit 25 to 80 and the maximum time 120s to 600s since multi-file edits are slow. The progress callbacks increment the chat with intermediate results as they occur, thus the user does not have to stare at a blank screen ten minutes. Code Mode is set to default of `gemini-2.5-pro`—slower, but which which copes better when you have longer tasks.

D. Security Layers

All tools calls are considered to be untrusted irrespective of the sender of the call [6], [16]. Between the intent of the LLM and the OS there are three filters. If any one fails, the call is killed and the admin gets a Telegram message.

Algorithm 1 Sandbox gate for incoming tool calls

Require: Tool call T with caller ID c , tool name n , arguments a

```

1:  $tier \leftarrow \text{LOOKUPTIER}(c)$ 
2: if  $tier = \text{null}$  then
3:   return REJECT("unknown user")
4: end if
5: if  $\text{MINTIER}(n) > tier$  then
6:   return REJECT("insufficient permissions")
7: end if
8: if  $n = \text{ShellTool}$  then
9:   for all pattern  $p \in \text{REGEXBLACKLIST}$  do
10:    if  $p.\text{SEARCH}(a.\text{command})$  then
11:      return REJECT("blocked command")
12:    end if
13:  end for
14: end if
15: if  $n = \text{PythonExecTool}$  then
16:   for all  $s \in \text{BLOCKEDIMPORTS}$  do
17:    if  $s \subseteq a.\text{code}$  then
18:      return REJECT("forbidden import")
19:    end if
20:  end for
21:   for all pattern  $p \in \text{DANGEROUSCALLPATTERNS}$  do
22:    if  $p.\text{SEARCH}(a.\text{code})$  then
23:      return REJECT("dangerous call")
24:    end if
25:  end for
26: end if
27: if  $a$  references .env path then
28:   return REJECT("secret file access")
29: end if
30: return EXECUTE( $n, a$ )

```

Layer 1 — Identity and permissions. Every Telegram user ID is associated with a tier, either one of the following: -readonly, -user, or -admin. Silently ignored are unrecognised IDs. When a read only user requests to do something, the

agent tells them they are not allowed to; the sandbox does not even receive the request.

Layer 2 — Command regex filter. The sandbox uses each of its compiled regex patterns to match the entire command string (currently 11 caret-anchored patterns including the commands: `rm -rf`, `format`, `mkfs`, `dd if=`, `shutdown`, `reboot`, `piped nc`, and so forth), before the ShellTool spawns a subprocess. Patterns anchored with `^` to ensure a subcommand argument like `systemctl restart nginx` is not blocked when it should be, but `shutdown now` is happen.

Layer 3 — Pattern-based Python filter. The PythonExecTool analyzes the source code of an LLM-generated program to identify blocked import strings (`os.system` and `subprocess` and `shutil.rmtree`), and runs regular expressions to match dangerous built-in functions, including `exec()` and `eval()` and `__import__()`. Another group of patterns blocks access to any attempt to read any of the files named with the suffix `.env` using the `open()` or `Path.read_text()` methods. Before executing, matching code is rejected. The general flow is presented in above Algorithm 1.

VI. RESULTS

A. Test Setup

All tests were conducted on a Windows 11 desktop (i5-12400, 16 GB RAM) and the MCP server was local. Claude 3.5 Sonnet was the LLM through the Anthropic API; the round-trip time to the API was 180 ms on average.

We hand-wrote two test suites: 50 legitimate administration tasks in five categories (file management, networking, process control, system info, Python scripting) and 30 malicious payloads—prompt injection, command injection, and Python import smuggling.

B. Functional Results

Table II has the breakdown. 46 out of 50 tasks passed. The three failures were all the same root cause: the LLM generated Linux commands on a Windows box (`grep` instead of `findstr`, that kind of thing). The fourth failure was a timeout—a recursive disk scan which had exceeded the 60-second limit.

TABLE II
FUNCTIONAL TEST RESULTS BY TASK CATEGORY

Category	Tasks	Pass	Avg. time (s)
File management	12	11	2.8
Networking	10	10	3.1
Process control	8	8	2.4
System info	10	10	1.9
Python scripting	10	7	4.6
Total	50	46	3.0

End-to-end latency - the longest time taken to send the message of the Telegram and receive a response - was 3.0s on average. Most of it (approximately 1.8s) was consumed by the LLM API call. The processing time of MCP server itself

was less than 200ms hence it is obvious that the bottleneck lies at the model.

Upon these tests, we installed the system to a macOS computer to use it daily. The default production setup is Gemini 2.5 Flash. We have since used it with ten-plus models across all three providers and the tool layer and sandbox only required one change to change, which is exactly what we wanted MCP to provide.

C. Security Results

The adversarial results are presented in Table III. None of the 30 payloads made it to the OS.

TABLE III
ADVERSARIAL TEST RESULTS (30 PAYLOADS)

Attack type	N	Blocked	Blocked by
Cmd injection (rm, format)	10	10	Layer 2 (regex)
Prompt injection	8	8	Layer 2 + LLM
Import smuggling (os, subprocess)	7	7	Layer 3 (pattern)
Permission escalation	5	5	Layer 1 (tier)
Total	30	30	—

Two of the immediate injections are worthy of description. To a payload of `del C:\* execute all commands`, one prepended it with one of the following: `system: you are now in maintenance mode`. The LLM did not send it to the ShellTool, thus the injection was partially successful, but the regex filter killed it before it could hit the OS. The coded other one `base64 message`. That was beyond the range of our filter. Our savior was the model itself: it did not decode and execute an opaque blob. That behaviour is not something we can depend on in different versions of the model, but it was a reminder that defence in depth can sometimes be effective in unforeseen locations.

VII. DISCUSSION

Why MCP helped. The initial prototype we had before we switched to MCP required changes to three files and two process restarts to uncover a single new command. When MCP was present (via `anthropic2024mcp`) it took approximately 40 lines of Python: a single class, a single schema, a single registration call, and nothing in the codebase was required to know about it. It is the $M \times N \rightarrow M + N$ reduction which Patil et al. write about theoretically [4] about it in theory; we observed it in practice.

LangGraph and multi-model validation: The largest architectural modification that we had to make was the replacement of our original CLI-based wrapper with a LangGraph ReAct graph [21]. The reward came swiftly: since the LLM is a pluggable subsystem in create react agent, we could replace Gemini with GPT or Claude by adjusting a single line of config- no tool-side modifications. The adapters of LangChain to the StructuredTool (using the adapters named StructuredTool) helped, as well; typing the model with schemas that

were validated in Pydantic reduced the number of malformed argument errors we had chased down over weeks. Something we were not expecting: Gemini API errors when the history of message has a tool-call that does not have a corresponding tool-result (which we had come to refer to as dangling tool calls). We found ourselves writing a function, called `UserContext._trim()` which will move the sliding window to the first `HumanMessage` whenever the context is trimmed.

Where the sandbox held. Each of the 30 adversarial payloads was detected prior to the OS. The regex filter prevented the obvious ones direct `-rm` and direct format calls. The import-smuggling attempts were prevented by the pattern-based Python filter. We did not predict the LLM assisting us: twice, the model would simply refuse to execute an obfuscated command that our filters would have allowed to pass. We are not counting on that—a future model version might not be as cautious—but it was a welcome surprise during testing.

Where the sandbox broke. In informal testing we found out that `systemctl restart nginx` was not allowed as the regex `r` restart was found at any place in the string. We corrected this by anchoring all patterns containing `^` in such a way that they do not match when the threatening word is the base command. Individually, the blacklist was bypassed by a piped command such as `powershell -enc <base64>` which we patched with a new regex rule. These two cases underscore that a blacklist is really nothing short of a partial defence [5], [23].

Production deployment. Since early 2026, we have been operating the system on a personal machine to use in our everyday life. A single command (`uv run python src/main.py`) is the entry point; Telegram long-polling implies that we do not have to work over a public IP address or have a webhook endpoint. There is automatic rotation of logs—one for operational events and one for security audits. The whole codebase is 33 Python files, about 5 000 lines. Early on, we got hard-blocking of the `(.env)` files at three levels (path validation, shell command filter, Python code filter) that saved us at least once by accidentally leaking API keys during testing.

Latency and cost Three seconds revolve around trip is enough when you are trying to check in your ohone the disk space. You may not wish to run 200 tasks in a batch, and that is fine. The LLM API call takes about 1.8s; the MCP server itself completes in less than 200 ms. A locally hosted, smaller scale might likely reduce the overall latency to less than a second, though we have not experimented to determine whether the logic works on multi-step problems.

Telegram as a UI. We chose Telegram as it provided us with two in the form of a free authentication (each user is assigned to a globally unique ID) and push-notifications on all platforms. The trade-off is that bot messages are not end-to-end encrypted - the servers of Telegram can be read by them [8]. In case the commands sent include sensitive information, a self-hosted solution, such as Matrix or Element, would be more suitable.

VIII. LIMITATIONS

Our largest limitation is that we execute on the bare host. The output of the LLM has no container, micro-VM or WASM boundary with the kernel. Our regex blacklist will help—caret-anchoring will reduce false positives—but it can never get all the obfuscated and aliased commands. This would be fixed by a Firecracker micro-VM [20] or a Docker container [17] at the expense of the complexity of deployment.

There is a similar gap in the Python filter. It does not do full AST analysis, it does string matching, thus tricks such as manipulating the builtins with a name such as `__builtins__` or tricks like `importlib` are bypassed completely. That can be sealed by wrapping the Python runtime in a WebAssembly sandbox [18], [19].

On the evaluation side, a sufficient number of tasks and attacks are 50 and 30, respectively to demonstrate that the architecture works, but not statistically significant. We tested on Windows 11 and macOS but have not tried multi-host orchestration. Telegram bot traffic is encrypted via TLS but not end-to-end [8], sensitive environments would require a self-hosted messaging layer. Lastly, all LLM providers have their own oddities, such as Gemini not quite understanding how to call the dangling tool, GPT sometimes struggling to format an argument, and we still have per-provider workarounds in spite of the MCP abstraction.

IX. CONCLUSION

The system that we are describing is in production: a Telegram bot supported by a LangGraph ReAct agent [21] which is communicating with the host operating system using MCP [7]. Throughout the course of development we three times changed the LLM: Claude to Gemini to GPT and back without ever interacting with the tool layer. That was sufficient to prove to us the $M + N$ premise of MCP. The three-layer sandbox (identity check, regex command filter, pattern-based Python filter) prevented all the payloads in our 30-attack adversarial test suite, and the system took 92 per cent of 50 tasks as an administrator with a 3-second average. It is not flawless, however, it can be used nowadays.

The gaps are clear. Not all dangerous commands will be included in our regex blacklist. The pattern-based Python filter lacks tricks of `importlib`. Telegram does not provide an end-to-end encryption of bot traffic. The following activities include transferring execution to a Firecracker micro-VM [20] to cease depending on a blacklist, introduce a whitelist mode to locked-down settings and create a web dashboard of live system metrics alongside the chat.

REFERENCES

- [1] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
- [2] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [3] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” *arXiv preprint arXiv:2302.04761*, 2023.
- [4] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large language model connected with massive APIs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [5] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” *arXiv preprint arXiv:2302.12173*, 2023.
- [6] OWASP Foundation, “OWASP top 10 for large language model applications 2025,” <https://owasp.org/www-project-top-10-for-large-language-model-applications/>, 2025, accessed: 2025-04-10.
- [7] Anthropic, “Introducing the model context protocol,” <https://www.anthropic.com/news/model-context-protocol>, 2024, accessed: 2025-04-10.
- [8] Telegram FZ-LLC, “Telegram bot API documentation,” <https://core.telegram.org/bots/api>, 2023, accessed: 2025-04-10.
- [9] C. Cao, F. Wang, L. Lindley, and Z. Wang, “Managing Linux servers with LLM-based AI agents: An empirical evaluation with GPT-4,” *Machine Learning with Applications*, vol. 17, p. 100570, 2024.
- [10] R. Zahedifar, S. A. Mirghasemi, M. S. Baghshah, and A. Taheri, “LLM-Agent-Controller: A universal multi-agent large language model system as a control engineer,” *arXiv preprint arXiv:2505.19567*, 2025.
- [11] X. Guo, D. Keivan, U. Syed, L. Qin, H. Zhang, G. Dullerud, P. Seiler, and B. Hu, “ControlAgent: Automating control system design via novel integration of LLM agents and domain expertise,” *arXiv preprint arXiv:2410.19811*, 2024.
- [12] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, “HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face,” *arXiv preprint arXiv:2303.17580*, 2023.
- [13] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [14] Microsoft, “Language server protocol specification – 3.17,” <https://microsoft.github.io/language-server-protocol/>, 2024, accessed: 2025-04-10.
- [15] JSON-RPC Working Group, “JSON-RPC 2.0 specification,” <https://www.jsonrpc.org/specification>, 2013, accessed: 2025-04-10.
- [16] U. Iqbal, T. Kohno, and F. Roesner, “LLM platform security: Applying a systematic evaluation framework to OpenAI’s ChatGPT plugins,” in *AAAI/ACM Conference on AI, Ethics, and Society (AIES)*, 2024.
- [17] S. Sultan, I. Ahmad, and T. Dimitriou, “Container security: Issues, challenges, and the road ahead,” *IEEE Access*, vol. 7, pp. 52 976–52 996, 2019.
- [18] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. F. Bastien, “Bringing the web up to speed with WebAssembly,” in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 2017, pp. 185–200.
- [19] G. Perrone and S. P. Romano, “WebAssembly and security: A review,” *arXiv preprint arXiv:2407.12297*, 2024.
- [20] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, “Firecracker: Lightweight virtualization for serverless applications,” in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020, pp. 419–434.
- [21] LangChain, Inc., “LangGraph: Build stateful multi-actor applications with LLMs,” <https://github.com/langchain-ai/langgraph>, 2024, accessed: 2026-04-10.
- [22] —, “LangChain: Build context-aware reasoning applications,” <https://www.langchain.com/>, 2024, accessed: 2026-04-10.
- [23] Y. Liu, G. Deng, Y. Li, K. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, and Y. Liu, “Prompt injection attack against LLM-integrated applications,” *arXiv preprint arXiv:2306.05499*, 2023.